

CHAPTER 10

有限狀態機器



在組合邏輯電路中，電路的輸出訊號只由目前的輸入訊號來決定。而在循序邏輯電路 (Sequential Circuit) 中，電路的輸出訊號除了由目前的輸入訊號決定外，也和先前的電路狀態有關。

簡單地說：組合邏輯電路是一種「無記憶性電路」 (Memory-Less Circuit)，而循序邏輯電路則是「有記憶性電路」。

在循序邏輯電路中，訊號的時序關係可分為：「同步循序邏輯電路」 (Synchronous Sequential Circuit) 與「非同步循序邏輯電路」 (Asynchronous Sequential Circuit)。



10.1 循序電路的基本模式

循序電路在數位系統的控制電路中扮演著重要的角色。循序電路 (又稱為 Deterministic Sequential Machine、Deterministic Finite Automation 或 Finite State Machine，有限狀態機器) 為一種具有記憶元件的邏輯電路。

最簡單實作一個有限狀態機器，可由下列二種方法來完成：

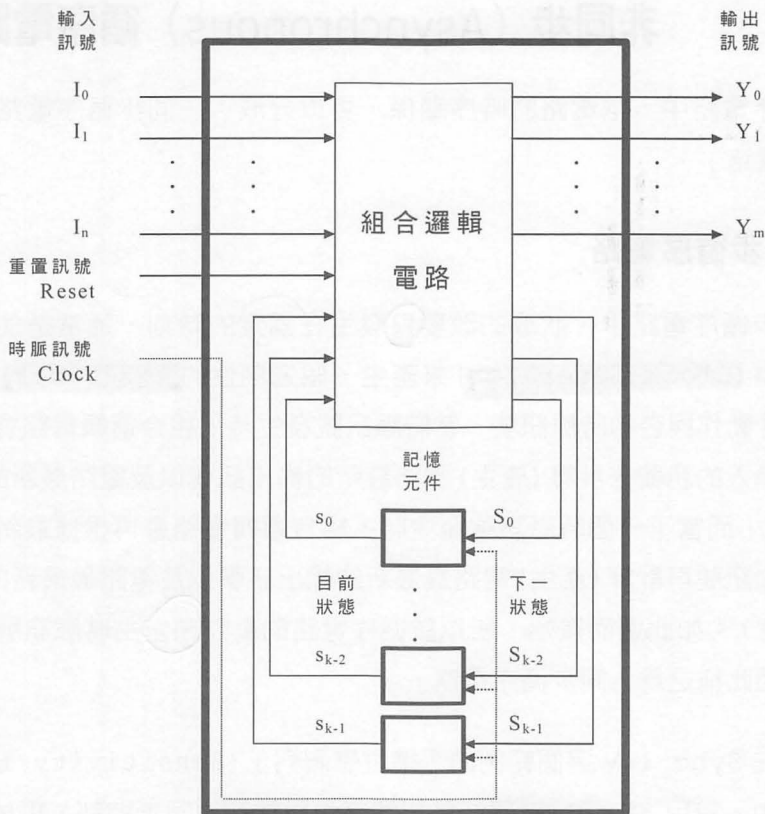
■ 一個唯讀記憶體 (ROM) 和一個計數器 (microcode 的方式)

採用這種設計的方式不但使得電路的面積較大，而且電路運作的速度也比較慢。因為是用 ROM 來控制電路的運作，所以被稱為微碼 (microcode) 控制的方式。

■ 組合邏輯電路 (random combinational logic) 加上記憶元件 (storage elements)

採用這種設計的方式在電路的面積及運作的速度都相對的較第一種採用 microcode 的方式來得佳。

循序電路的基本電路架構，如圖「循序電路的基本電路架構」所示。循序電路是由 n 個獨立的輸入變數 ($I_0, I_1, \dots, I_n$)、 m 個獨立的輸出變數 ($Y_0, Y_1, \dots, Y_m$) 及 k 個記憶元件所組成。記憶元件的輸入訊號組合 ($S_0, S_1, \dots, S_{k-1}$) 稱為該循序電路的目前狀態 (Present State, PS)，而其輸出訊號組合 ($S_0, S_1, \dots, S_{k-1}$) 稱為下一個狀態 (Next State, NS)。將目前狀態 (Present State, PS) 與輸入訊號 ($I_0, I_1, \dots, I_{n-1}$) 組合後，決定該循序電路的輸出訊號 ($Y_0, Y_1, \dots, Y_{m-1}$) 與下一個狀態 (Next State, S)。重置 (Reset) 訊號用於將此循序電路的下一狀態設定在某一個電路的狀態，時脈 (Clock) 訊號則是用於此循序電路的運作時脈。



▲ 循序電路的基本電路架構

循序電路的狀態由目前狀態轉變到下一個狀態的現象稱為「狀態的轉移」(State Transition)，簡稱為「轉態」。

對循序電路而言、每一記憶元件是一個狀態變數(State Variable)，有 0 與 1 兩種不同的值。具有 k 個記憶元件的循序電路，最多有 2^k 種不同的狀態。也就是說如果某個循序電路有 R 個不同的狀態，那麼該電路所需要的記憶元件至少為 $2^k \geq R$ ，其中： $k \geq \lceil \log_2 R \rceil$ 。



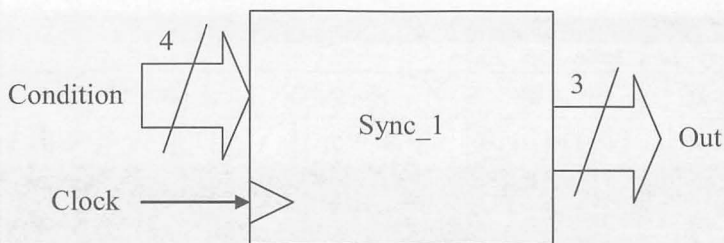
10.2 同步 (Synchronous) 循序電路與 非同步 (Asynchronous) 循序電路

在循序電路中，依電路的時序關係，可以分成：「同步循序電路」與「非同步循序電路」。

(一) 同步循序電路

在同步循序電路中，狀態的改變只發生在固定的時刻，通常是使用一個時脈產生電路 (Clock Generator) 來產生一個週期性的脈波 (Clock)，作為控制記憶元件動作與否的時脈訊號。當時脈訊號發生時，組合邏輯電路會根據目前的狀態及輸入的訊號去計算 (產生) 電路最新的輸出訊號以及電路最新的狀態 (下一個狀態)。而當下一個時脈訊號發生時，組合邏輯電路會再根據最新的狀態及最新輸入的訊號再計算 (產生) 電路最最新的輸出訊號以及電路最最新的狀態 (下一個狀態)；如此週而復始。所以該循序電路的動作完全由時脈訊號所作同步的控制，因此稱之為「同步循序電路」。

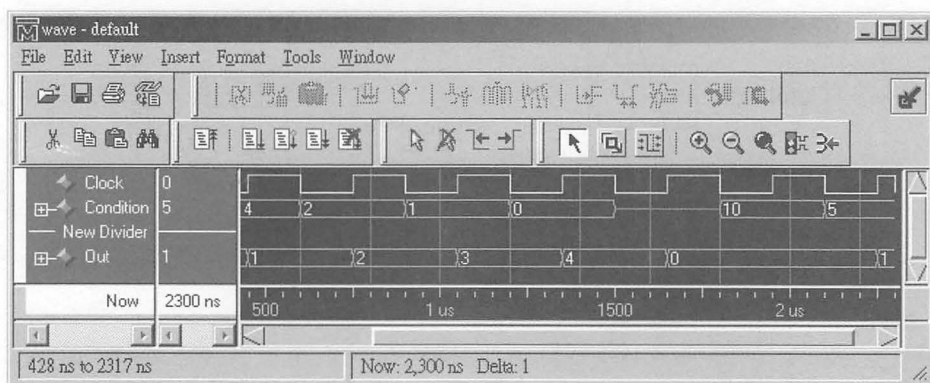
範例 在 Sync_1.V 這個範例的「準位感測列」(Sensitivity list, @) 中，只用 Clock 這個時脈訊號來作為電路的同步控制，再依據 Condition 這個輸入訊號來決定輸出訊號值。



```

// Sync_1.V: 同步循序電路範例
module Sync_1_machine (Clock, Condition, Out);
input      Clock;
input  [3:0] Condition;
output  [2:0] Out;
reg     [2:0] Out;

always @(posedge Clock) // 時脈訊號，用於電路的同步控制
begin
    casex (Condition) // 依據 Condition 這個輸入訊號來決定輸出訊號值
        4'b1???:
            begin
                Out = 3'b000;
            end
        4'b?1??:
            begin
                Out = 3'b001;
            end
        4'b??1?:
            begin
                Out = 3'b010;
            end
        4'b???1:
            begin
                Out = 3'b011;
            end
        4'b0000:
            begin
                Out = 3'b100;
            end
        default:
            begin
                Out = 3'b111;
            end
    endcase
end
endmodule
  
```



▲ Sync_1.V 合成後，電路的模擬結果

(二) 非同步循序電路

在非同步循序電路中，電路的行為只受到輸入訊號的改變次序所影響，而且可以在任何時刻發生，也就是不需要週期性的脈波 (Clock) 來決定電路的運作時機。由於電路中沒有時脈裝置來作同步的控制電路運作時機，非同步循序電路能否正常的動作完全取決於電路的輸入訊號。讓非同步循序電路正確地運作最簡單的方式就是限制：輸入訊號的改變時機，當電路達到穩定的狀態之後才能改變新的輸入訊號值。

範例 在 Async_1.V 這個範例的「準位感測列」(Sensitivity list, @) 中，當「Clock」這個時脈訊號或者是「Condition」這個輸入訊號有被改變，則電路便會依據 Condition 這個輸入訊號來決定輸出訊號值。

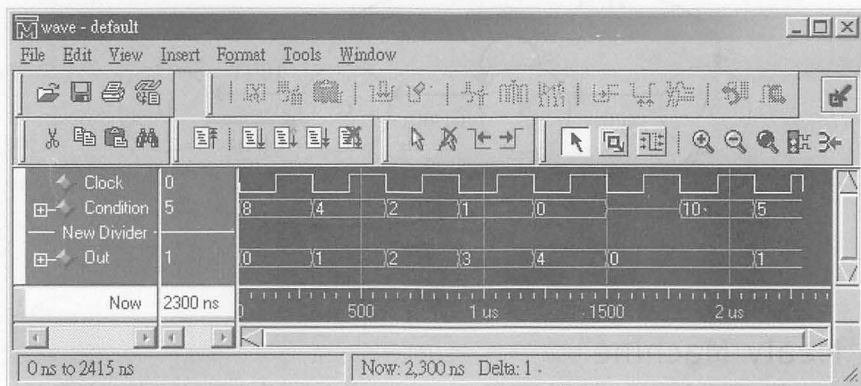
```
// Async_1.V: 非同步循序電路範例
module Async_1_machine (Clock, Condition, Out);
input      Clock;
input      [3:0] Condition;
output     [2:0] Out;
reg        [2:0] Out;
```



```

always @(Clock or Condition) // 非同步:因為有 Clock 及 Condition 訊號
begin
    casex (Condition)
        4'b1???:
            begin
                Out = 3'b000;
            end
        4'b?1??:
            begin
                Out = 3'b001;
            end
        4'b??1?:
            begin
                Out = 3'b010;
            end
        4'b???1:
            begin
                Out = 3'b011;
            end
        4'b0000:
            begin
                Out = 3'b100;
            end
        default:
            begin
                Out = 3'b111;
            end
    endcase
end
endmodule

```



▲ Async_1.V 合成後，電路的模擬結果



10.3 有限狀態機器 (Finite State Machine, FSMs) 的簡介

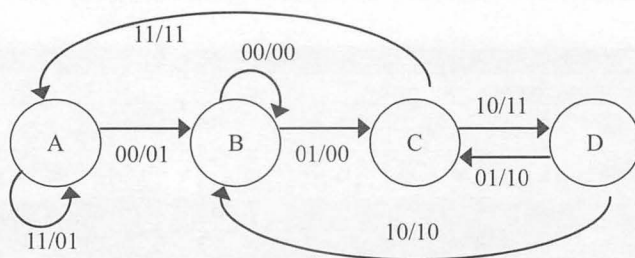
「有限狀態機器」(Finite State Machine, FSMs) 簡稱：「有限狀態機」，是循序電路中是變化性最多的一種循序邏輯電路，通常用在比較特殊要求的控制電路中。而在前一章所提介紹的：暫存器、移位暫存器、計數器... 等等均可以看成是一種功能固定的有限狀態機器。

10.3.1 有限狀態機的種類

有限狀態機器依其輸入的訊號、輸出的訊號，以及狀態之間的關係，可細分為二大類：Moore machine 及 Mealy Machine。

(一) Moore Machine

這種有限狀態機器其輸出的訊號只與目前的狀態有關，而與輸入的訊號無關。以「狀態圖 1：Moore machine」的狀態 A 來說，不管輸入訊號給 00 或是 11，輸出訊號皆會是 01。

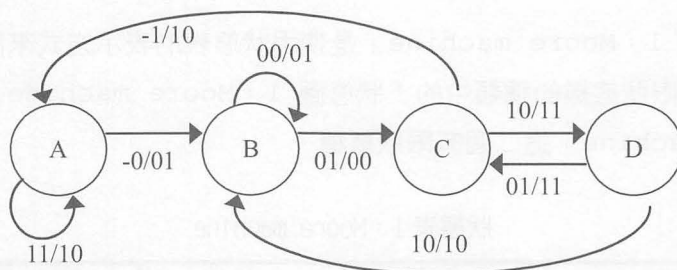


▲ 狀態圖 1：Moore machine

(二) Mealy Machine

這種有限狀態機器其輸出的訊號需考慮目前的狀態及輸入的訊號。Mealy

Machine 雖然可以用較少的狀態變數來實現一個有限狀態機器，但其輸出訊號會受到輸入訊號的干擾，因而容易在輸出訊號上產生「突波」現象；如果延長時脈的週期，讓電路有較長的時間處於穩態的狀況，相對的發生突波的時間就顯得小多了。



▲ 狀態圖 2：Mealy machine

10.3.2 有限狀態機的表示方式

通常我們可以使用一些較有系統的方法，像是：「邏輯方程式」（交換函數）、「狀態轉移圖」（State Transition Graph, STG）簡稱為：「狀態圖」、「狀態轉移表」（State Transition Table, STT）簡稱為：「狀態表」、「時序序列」（Timing Sequences）、以及「時序圖」（Timing Diagrams）來描述移位暫存器、計數器或者是特定狀態機器…等等的「有限狀態機器」，用「狀態圖」來設計所需要的有限狀態機器。

（一）狀態圖（State Transition Graph, STG）

在狀態圖中：每一個頂點（圓圈）代表循序電路中的一個狀態，頂點（圓圈）內的符號代表該狀態的名稱，每一個有方向性的分支（Directed Branch）代表循序電路的一個狀態轉移，有方向性的分支旁的標記代表該電路轉移所對應的輸入訊號的變化與輸出訊號的變化。例如：在上一小節所討論的有限狀態機的種類中，「狀態圖 1：Moore machine」及「狀態圖 2：Mealy machine」均是

使用狀態圖的方式來表示所需要的有限狀態機器。

(二) 狀態表 (State Transition Table, STT)

在狀態表中：每一列相當於是在狀態圖中所包含的一個狀態轉移動作。

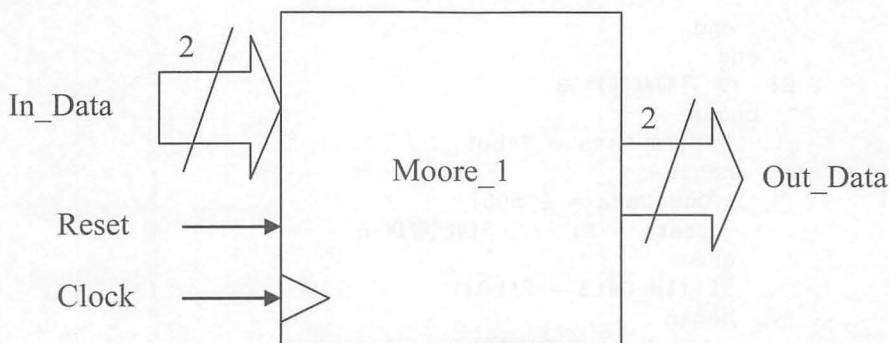
「狀態表 1 : Moore machine」是使用狀態表的表示方式來說明在上一小節所討論的有限狀態機的種類中的「狀態圖 1 : Moore machine」及「狀態圖 2 : Mealy machine」這二個有限狀態機。

狀態表 1 : Moore machine

輸入訊號	目前狀態	下一狀態	輸出訊號
11	A	A	01
00	A	B	01
00	B	B	00
01	B	C	00
10	C	D	11
11	C	A	11
10	D	B	10
01	D	C	10

「狀態表 1 : Moore machine」所顯示的例子為在上一小節所討論的有限狀態機的種類中的「狀態圖 1 : Moore machine」，這個有限狀態機包含了：2 個輸入訊號、2 個輸出訊號及 4 種狀態，在表中 A、B、C 及 D 這四個符號代表著四種不同的狀態。這個表格被切分為四個欄位，分別為：輸入訊號、目前狀態、下一狀態及輸出訊號。

從這個表格中，可以很明顯地看出：不管目前狀態為何，它的輸出訊號一定相同。這點說明了：Moore machine 輸出的訊號只與目前的狀態有關，而與輸入的訊號無關。



▲ 狀態表 1：Moore machine 的 Verilog 電路描述之邏輯符號

```
// Moore_1.V: 狀態表 1: Moore machine 的 Verilog 電路描述
module Moore_1 (Clock, Reset, In_Data, Out_Data);
input      Clock, Reset;
input  [1:0] In_Data;
output  [1:0] Out_Data;
reg     [1:0] Out_Data;
reg     [1:0] state;

parameter A = 2'b00, B = 2'b01, C = 2'b10, D = 2'b11;

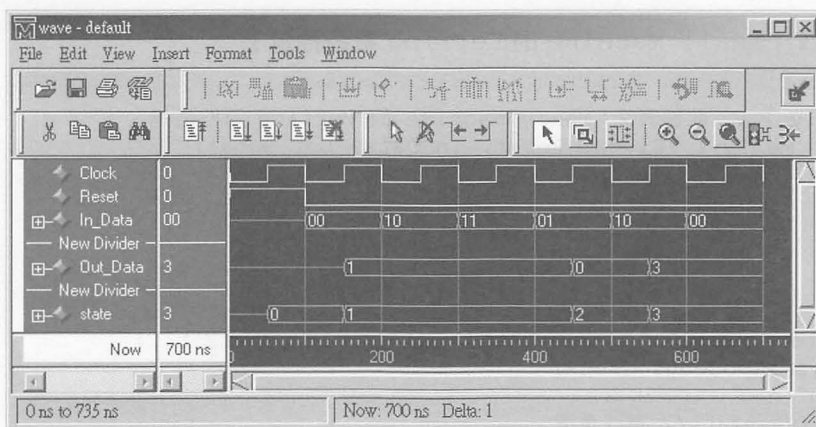
always @(posedge Clock)
begin
    if (Reset) // 電路重置
    begin
        state = A; // 初始狀態設為 A
    end
    else
    begin
        case (state)
            A: // 目前狀態為 A
            begin
                if (In_Data = 2'b11)
                begin
                    Out_Data = 2'b01;
                    state = A; // 下個狀態為 A
                end
                if (In_Data = 2'b00)
                begin
                    Out_Data = 2'b01;
                    state = B; // 下個狀態為 B
                end
            end
        end
    end
end
```

```

        end
    end
B: // 目前狀態為 B
begin
    if (In_Data = 2'b00)
    begin
        Out_Data = 2'b00;
        state = B; // 下個狀態為 B
    end
    if (In_Data = 2'b01)
    begin
        Out_Data = 2'b00;
        state = C; // 下個狀態為 C
    end
end
C: // 目前狀態為 C
begin
    if (In_Data = 2'b10)
    begin
        Out_Data = 2'b11;
        state = D; // 下個狀態為 D
    end
    if (In_Data = 2'b11)
    begin
        Out_Data = 2'b11;
        state = A; // 下個狀態為 A
    end
end
D: // 目前狀態為 D
begin
    if (In_Data = 2'b10)
    begin
        Out_Data = 2'b10;
        state = B; // 下個狀態為 B
    end
    if (In_Data = 2'b01)
    begin
        Out_Data = 2'b10;
        state = C; // 下個狀態為 C
    end
end
    endcase
end
end

endmodule

```



▲ Moore_1.V 合成後，電路的模擬結果

狀態表 2：Mealy machine

輸入訊號	目前狀態	下一狀態	輸出訊號
00	A	B	01
11	A	A	10
00	B	B	01
01	B	C	00
10	C	D	11
01	C	A	10
10	D	B	10
01	D	C	11

「狀態表 2：Mealy machine」所顯示的例子為在上一小節所討論的有限狀態機種類中的「狀態圖 2：Mealy machine」，這個有限狀態機包含了：2 個輸入訊號、2 個輸出訊號及 4 種狀態，在表中 A、B、C 及 D 這四個符號代表著四種不同的狀態。這個表格被切分為四個欄位，分別為：輸入訊號、目前狀態、下一狀態及輸出訊號。

從這個表格中，可以很明顯地看出：Mealy machine 輸出的訊號必須考慮目前的狀態及輸入的訊號。

【狀態表 2 : Mealy machine 的 Verilog 電路描述】

```
// Mealy_1.V : 狀態表 2 : Mealy machine 的 Verilog 電路描述
module Mealy_1 (Clock, Reset, In_Data, Out_Data);
input      Clock, Reset;
input      [1:0] In_Data;
output     [1:0] Out_Data;
reg        [1:0] Out_Data;
reg        [1:0] state;

parameter A = 2'b00, B = 2'b01, C = 2'b10, D = 2'b11;
always @(posedge Clock)
begin
    if (Reset) // 電路重置
    begin
        state = A; // 初始狀態設為 A
    end

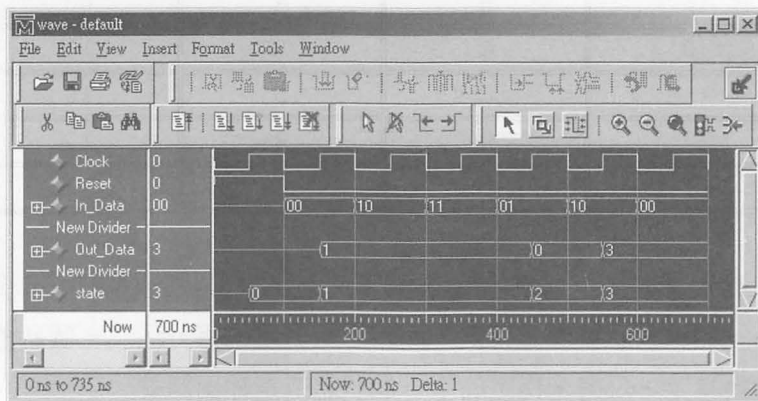
    else
    begin
        case (state)
            A: // 目前狀態為 A
                Begin
                    if (In_Data == 2'bx0)
                    begin
                        Out_Data = 2'b01;
                        state = B; // 下個狀態為 B
                    end
                    if (In_Data == 2'b11)
                    begin
                        Out_Data = 2'b10;
                        state = A; // 下個狀態為 A
                    end
                end
            B: // 目前狀態為 B
                begin
                    if (In_Data == 2'b00)
                    begin
                        Out_Data = 2'b01;
                        state = B; // 下個狀態為 B
                    end
                    if (In_Data == 2'b01)
                    begin
                        Out_Data = 2'b00;
                        state = C; // 下個狀態為 C
                    end
                end
        end
    end
end
```



```

C: // 目前狀態為 C
begin
    if (In_Data == 2'b10)
    begin
        Out_Data = 2'b11;
        state = D; // 下個狀態為 D
    end
    if (In_Data == 2'b11)
    begin
        Out_Data = 2'b10;
        state = A; // 下個狀態為 A
    end
end
D: // 目前狀態為 D
begin
    if (In_Data == 2'b10)
    begin
        Out_Data = 2'b10;
        state = B; // 下個狀態為 B
    end
    if (In_Data == 2'b01)
    begin
        Out_Data = 2'b11;
        state = C; // 下個狀態為 C
    end
end
endcase
end
end
endmodule

```



▲ Mealy_1.V 合成後，電路的模擬結果

10.3.3 完全指定狀態 (Completely Specified) 與 不完全指定狀態 (Incompletely Specified)

在本章中的第一小節「循序電路的基本模式」中我們說明了：具有 k 個記憶元件的循序電路，最多有 2^k 種不同的狀態。也就是說，如果某個循序電路有 r 個不同的狀態，那麼該電路所需要的記憶元件至少為 $2^k \geq R$ ，其中： $k \geq \lceil \log_2 R \rceil$ 。

若在一個循序電路任一個狀態 S_a 它所對應的每一組輸入訊號都有指定，而且它所對應的每一個下一個狀態與輸出訊號（輸出函數）均有明確地指定，則稱為「完全指定狀態」(Completely Specified)。反之，若有部份任一個狀態 S_b 它所對應的部份每一組輸入訊號、每一個下一個狀態或輸出訊號（輸出函數）未加以給定，則稱為「不完全指定狀態」(Incompletely Specified)。

「完全指定狀態」表 1

輸入訊號	目前狀態	下一狀態	輸出訊號
0	A	B	01
1	A	C	10
0	B	D	01
1	B	A	00
0	C	A	11
1	C	D	10
0	D	B	10
1	D	C	11

「完全指定狀態」表 1 所顯示的例子中的任一個狀態 (A、B、C 或 D)，它所對應的每一組輸入訊號都有指定，而且它所對應的每一個下一個狀態與輸出訊號均有明確地指定，沒有未指定的下一個狀態或是輸出訊號。

【「完全指定狀態」表 1 的 Verilog 電路描述】

```
// FSM_CS.V: 「完全指定狀態」表 1
module FSM_CS (Clock, Reset, In_Data, Out_Data);
input      Clock, Reset;
input      In_Data;
output [1:0] Out_Data;
reg [1:0] Out_Data;

reg [1:0] state;

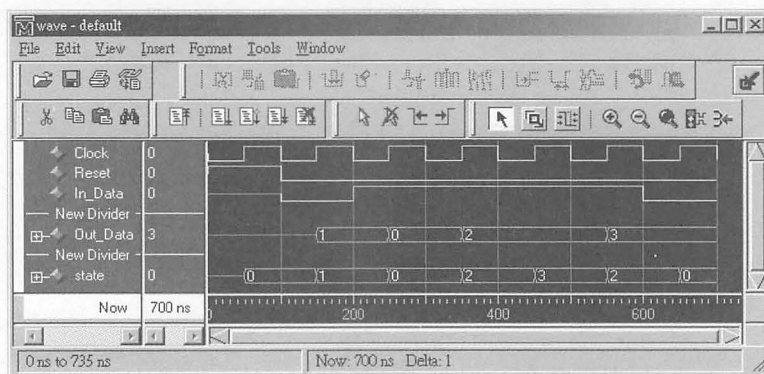
parameter A = 2'b00, B = 2'b01, C = 2'b10, D = 2'b11;

always @(posedge Clock)
begin
    if (Reset) // 電路重置
    begin
        state = A; // 初始狀態設為 A
    end
    else
    begin
        case (state)
            A: // 目前狀態為 A
            begin
                if (In_Data == 1'b0)
                begin
                    Out_Data = 2'b01;
                    state = B; // 下個狀態為 B
                end
                if (In_Data == 1'b1)
                begin
                    Out_Data = 2'b10;
                    state = C; // 下個狀態為 C
                end
                if (In_Data == 1'b1)
                begin
                    Out_Data = 2'b11;
                    state = D; // 下個狀態為 D
                end
            end
            B: // 目前狀態為 B
            begin
                if (In_Data == 1'b0)
                begin
                    Out_Data = 2'b01;
                    state = D; // 下個狀態為 D
                end
                if (In_Data == 1'b1)
                begin
                    Out_Data = 2'b00;
                    state = A; // 下個狀態為 A
                end
            end
        endcase
    end
end
```

```

end
C: // 目前狀態為 C
begin
    if (In_Data == 1'b0)
    begin
        Out_Data = 2'b11;
        state = A; // 下個狀態為 A
    end
    if (In_Data == 1'b1)
    begin
        Out_Data = 2'b10;
        state = D; // 下個狀態為 D
    end
end
D: // 目前狀態為 D
begin
    if (In_Data == 1'b0)
    begin
        Out_Data = 2'b10;
        state = B; // 下個狀態為 B
    end
    if (In_Data == 1'b1)
    begin
        Out_Data = 2'b11;
        state = C; // 下個狀態為 C
    end
end
endcase
end
end
endmodule

```



▲ FSM_CS.V 合成後，電路的模擬結果

「不完全指定狀態」表 2

輸入訊號	目前狀態	下一狀態	輸出訊號
0	A	B	01
1	A	—	10
0	B	D	01
1	B	A	-0
0	C	A	11
1	C	—	1-
0	D	B	-0
1	D	C	11

「不完全指定狀態」表 2 所顯示的例子中：

- 當目前狀態為 A 且輸入訊號為 1 時，它的下一狀態為「—」（沒有指定下一個狀態）。同理：目前狀態為 C 且輸入訊號為 1 時，也未指明它的下一狀態。
- 當目前狀態為 B 且輸入訊號為 1 時，它的輸出訊號（輸出函數）為「-0」（沒有指定輸出訊號）。同理：目前狀態為 C 且輸入訊號為 1，以及目前狀態為 D 且輸入訊號為 0，也都未指明它的輸出訊號。

以上情形均符合了：部份每一個下一個狀態或輸出訊號（輸出函數）未加以指定，則稱為「不完全指定狀態」（Incompletely Specified）。

【「不完全指定狀態」表 2 的 Verilog 電路描述】

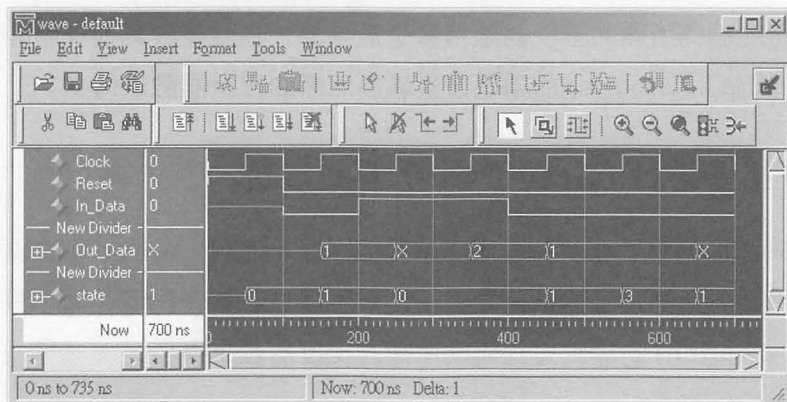
```
// FMS_IS.V：「不完全指定狀態」表 2
module FMS_IS (Clock, Reset, In_Data, Out_Data);
input      Clock, Reset;
input      In_Data;
output [1:0] Out_Data;
reg [1:0] Out_Data;
reg [1:0] state;
parameter A = 2'b00, B = 2'b01, C = 2'b10, D = 2'b11;
```

```
always @(posedge Clock)
begin
    if (Reset) // 電路重置
    begin
        state = A; // 初始狀態設為 A
    end
    else
    begin
        case (state)
            A: // 目前狀態為 A
            begin
                if (In_Data == 1'b0)
                begin
                    Out_Data = 2'b01;
                    state = B; // 下個狀態為 B
                end
                if (In_Data == 1'b1)
                begin
                    Out_Data = 2'b10;
                    // 下個狀態為 XX
                end
            end
            B: // 目前狀態為 B
            begin
                if (In_Data == 1'b0)
                begin
                    Out_Data = 2'b01;
                    state = D; // 下個狀態為 D
                end
                if (In_Data == 1'b1)
                begin
                    Out_Data = 2'bx0;
                    state = A; // 下個狀態為 A
                end
            end
            C: // 目前狀態為 C
            begin
                if (In_Data == 1'b0)
                begin
                    Out_Data = 2'b11;
                    state = A; // 下個狀態為 A
                end
                if (In_Data == 1'b1)
                begin
                    Out_Data = 2'b1x;
                    // 下個狀態為 XX
                end
            end
        end
    end
end
```

```

D: // 目前狀態為 D
begin
  if (In_Data == 1'b0)
  begin
    Out_Data = 2'b00;
    state = B; // 下個狀態為 B
  end
  if (In_Data == 1'b1)
  begin
    Out_Data = 2'b11;
    state = C; // 下個狀態為 C
  end
end
End
endcase
end
end
endmodule

```



▲ FMS_IS.V 合成後，電路的模擬結果

10.3.4 有限狀態機的編碼方式 (State Assignment 或 State Encoding)

適當地選擇有限狀態機器狀態的編碼方式：可以讓此有限狀態機器中之組合邏輯電路的複雜度降低 (也是電路面積會減小)，同時也會影響到正反器所需的數目 (如果正反器所需的數目愈少，電路面積也就會愈小)。

最佳的狀態編碼方式這個問題已經被證明為：NP-Complete (不能用一個數學理論上的多項式方程式來表示這個問題的複雜度)。許多的實驗結果顯示出：即使是使用同樣的一種編碼方式，對於任一個電路中的狀態都用排列組合的方式給予不同的狀態編碼，將會得到不同的電路面積。

假設 f 為正反器所需的數目、 S 為狀態的數目，則： $2^f \leq S \leq 2^{f-1}$ ，最多的狀態編碼方式為： $(2^f!) / (2^f - S)!$ 種編碼方式。

有限狀態機器依其狀態的編碼方式，我們列出了下列幾種方式：

(一)「二進制碼」(Binary Code) 編碼方式

二進制碼編碼方式同樣也是一種最小長度 (minimal length) 的編碼方式，所以用於編碼所需的正反器數目會是最少的，也就是較其他採用非最小長度 (Non-minimal length) 的編碼方式編碼所需的正反器數來得少。

(二)「格雷碼」(Gray Code) 編碼方式

格雷碼編碼方式同樣也是一種最小長度 (minimal length) 的編碼方式，所以用於編碼所需的正反器數目是最少的，也就是較其他採用非最小長度 (Non-minimal length) 的編碼方式編碼所需的正反器數來得少。

在格雷碼編碼方式中，二個相鄰的狀態被選擇的編碼只有一個位元是不同的，也就是用在電路中，只有一個正反器的值會發生變化，因此如果先分析電路狀態的轉移關係再配合使用這種編碼方式，可以讓電路的消耗功率較低。

(三)「One-Hot 碼」編碼方式

One-Hot 碼編碼方式是一種非最小長度 (Non-minimal length) 的編碼方式，所以用於編碼所需的正反器數目較其他採用最小長度 (minimal length) 的編碼方式編碼所需的正反器數來得多，而且所需的正反器數目就等於狀態機的

狀態數目。

在 One-Hot 碼編碼方式中，不管電路目前是在那一個狀態，只有一個正反器的值是有動作的（也就是只有一個為 1，其他為 0；或者是只有一個為 0，其他為 1）。因此可以利用這種特性增加一個檢查「狀態」（正反器的值是不是只有一個為 1，其他為 0）的電路，如果電路的「狀態」為非本電路的正反器狀態值，則表示電路在轉態的過程中發生了錯誤。檢查「狀態」的電路便會送出一個「狀態錯誤」的訊號給外界，告訴外界此狀態機發生問題了，讓接受此「狀態錯誤」訊號的電路去處理相對的動作。

什麼是電路的「狀態」為非本電路的正反器狀態值？

比如說像是：跳到了一個正反器的值是 1 的個數不對，或者是跳到了一個正反器的值雖然只有一個為 1，但電路中沒有設計會用到此種狀態…等等狀況。

例 1 假設某個有限狀態機有 3 種狀態，分別為 S1、S2 及 S3，編碼為 $S1 = 3'b001$ 、 $S2 = 3'b010$ 、 $S3 = 3'b100$ 。當 S1 轉態至 S2 時發生了錯誤，正反器的值為 011（或者是 000、101、110、111…等等）的這種情形。

例 2 假設某個有限狀態機有 3 種狀態，分別為 S1、S2 及 S3，編碼為 $S1 = 4'b0001$ 、 $S2 = 3'b0010$ 、 $S3 = 4'b0100$ ，和上個例子不同的是使用 4 個正反器用於狀態的編碼，而且 $4'b1000$ 也是 4 個位元的 One-Hot 碼之一，但是這個例子中沒有使用到。當 S1 轉態至 S2 時發生了錯誤，也就是正反器的值為 0011（或者是 0000、0101、0110、0111、1001、1010、1011、1100、1101、1110、1111…等等）的這種情形。

這種檢查電路的缺點，為何？

檢查電路無法檢查出跳到了一個電路的某個有被用到的狀態（正反器的值只有 1 個為 1），但這個轉態動作是希望跳到另一個狀態，也就是跳到別的狀態的這種狀況。

例 3 假設某個有限狀態機有 3 種狀態，分別為 S1、S2 及 S3，編碼為 S1 = 3'b001、S2 = 3'b010、S3 = 3'b100。當 S1 轉態至 S2 時發生了錯誤，正反器的值為 100，也就是跳到了 S3 這個狀態，像這種錯誤就無法被檢查出來。

（四）「K-Hot 碼」編碼方式

K-Hot 碼是一種非最小長度 (Non-minimal length) 的編碼方式，所以用於編碼所需的正反器數目較其他採用最小長度 (minimal length) 的編碼方式編碼所需的正反器數來得多。又稱為 (k, p) 碼，一般的唸法為 k 取 p 碼。

One-Hot 碼為 K-Hot 碼的一種特例，因為 One-Hot 碼為：狀態變數的正反器值有 1 的數目為 1 個，用於編碼所需的正反器數目為 n 個，總共可用於編碼的數目有 $C(n, 1) = n! / (1! * (n-1)!) = n! / (n-1)!$ 個。

在 K-Hot 碼中，假設：r 為狀態變數的正反器值有 1 的數目，n 為用於編碼所需的正反器數目，那麼總共可用於編碼的數目有 $C(n, r) = n! / (r! * (n-r)!) = n! / (r! * (n-r)!)$ 個。

例 4 假設狀態變數的正反器值有 1 的數目為 2 個，用於編碼所需的正反器數目為 4 個，那麼總共可用於編碼的數目有 $C(4, 2) = 4! / (2!) * (2!) = 4 * 3 * 2 * 1 / (2 * 1) * (2 * 1) = 6$ 個。

包括那 6 個呢？

1. 0011、2. 0101、3. 1001、4. 0110、5. 1010、6. 1100。

例 5

假設：狀態變數的正反器值有 1 的數目為 2 個，用於編碼所需的正反器數目為 5 個，那麼總共可用於編碼的數目有 $C(5, 2) = 5! / (2! * (3!)) = 5 * 4 * 3 * 2 * 1 / (2 * 1) * (3 * 2 * 1) = 10$ 個。

包括那 10 個呢？

1. 000**11**、2. 00**101**、3. 0**1001**、4. **10001**、5. 00**110**、
6. 0**1010**、7. **10010**、8. 0**1100**、9. **10100**、10. **11000**。

例 6

假設：狀態變數的正反器值有 1 的數目為 2 個，用於編碼所需的正反器數目為 6 個，那麼總共可用於編碼的數目有 $C(6, 2) = 6! / (2! * (4!)) = 6 * 5 * 4 * 3 * 2 * 1 / (2 * 1) * (4 * 3 * 2 * 1) = 15$ 個。

包括那 15 個呢？

聰明的您，請先自行推算吧。如果覺得不好推，那就請您從底下排出來的結果來找出關係：只要先固定最右邊的 1，然後一直由右向左插入 1 (其他的地方給 0)，再固定由右數的第 2 個 1；一樣由右向左插入 1 (其他的地方給 0)，重覆上述步驟，直到最左的二個位置為 11，其餘為 0 的這個碼為止。

1. 0000**11** 6. 000**110** 10. 00**1100** 13. 0**11000** 15. **110000**
2. 000**101** 7. 00**1010** 11. 0**10100** 14. **101000**
3. 00**1001** 8. 0**10010** 12. **100100**
4. 0**10001** 9. **100010**
5. **100001**

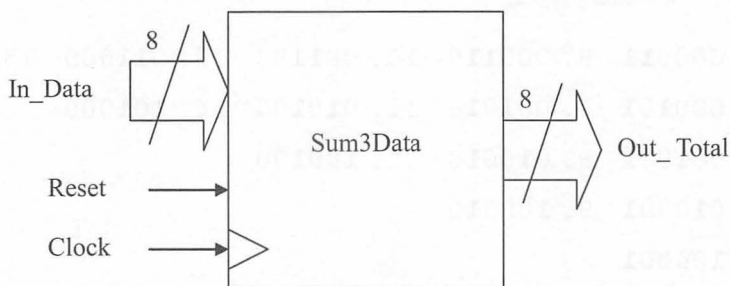
各種有限狀態機器其狀態編碼方式的比較表

(假設某個有限狀態機有 8 種狀態，分別為：S1、S2、S3、S4、S5、S6、S7 及 S8)

狀態名稱	二進制碼	格雷碼	One-Hot 碼	(5, 2)-碼 (K-Hot 碼)
	編碼所需的 正反器數目 3 個	編碼所需的 正反器數目 3 個	編碼所需的 正反器數目 8 個	編碼所需的 正反器數目 5 個
S1	000	000	00000001	00011
S2	001	001	00000010	00101
S3	010	011	00000100	01001
S4	011	010	00001000	10001
S5	100	110	00010000	00110
S6	101	111	00100000	01010
S7	110	101	01000000	10010
S8	111	100	10000000	01100

註：此表中的 (5, 2) 碼，尚有 10100 及 11000 可以被選作某個狀態名稱的編碼。

這些碼怎麼樣應用到我們的 Verilog 電路描述中呢？請看下面這個範例。



▲ 各種有限狀態機器其狀態編碼方式的 Verilog 電路描述，範例 Sum3Data.V 之邏輯符號

在這個有限狀態機中一共有 3 個狀態，分別為 s0、s1 及 s2；只要從 **reg [1:0] state;** 及 **parameter s0=2'b00, s1=2'b01, s2=2'b10;** 這兩行來設定採用什麼樣的狀態編碼方式，以及相對用來記錄有限狀態機內部狀

態所需要的正反器 (也就是 reg) 數目。

值得一提的是：如果要將 Sum3Data.V 這個 Verilog 電路描述檔由範例中的二進制碼編碼方式改為其他的編碼方式，除了需要更改 `reg [1:0] state;` 及 `parameter s0=2'b00, s1=2'b01, s2=2'b10;` 這二行描述以外，這個 Verilog 電路描述的其他部份完全不用更改，所以改變有限狀態機器其狀態編碼方式是很容易的。

- 採用二進制碼編碼方式 (00, 01, 10)，3 個狀態需 2 個正反器 (也就是 `reg [1:0]`)。
- 若改用 One-Hot 碼編碼方式 (001, 010, 100)，則需 3 個正反器 (`reg [2:0]`)。只要將 `reg [1:0] state;` 更改為 `reg [2:0] state;` 及 `parameter s0=2'b00, s1=2'b01, s2=2'b10;` 更改為 `parameter s0=3'b001, s1=3'b010, s2=3'b100;` 即可。
- 若選用 (3, 2)-碼編碼方式 (011, 101, 110)，同樣是需要 3 個正反器 (`reg [2:0]`)。只要將 `reg [1:0] state;` 更改為 `reg [2:0] state;` 及 `parameter s0=2'b00, s1=2'b01, s2=2'b10;` 更改為 `parameter s0=3'b011, s1=3'b101, s2=3'b110;` 即可。

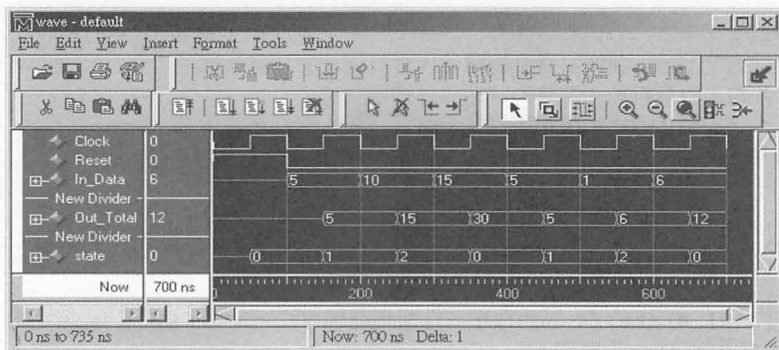
【範例 Sum3Data.V 的 Verilog 電路描述】

```
// Sum3Data.V
// 每當有一個 Clock 訊號來時就將輸入的資料值加至計數值中，
// 連加三個輸入的資料後、電路的狀態改為初始狀態，
// 也就是當下一個 Clock 訊號來時計數值改為輸入的資料值。
module Sum_3_Continuous_Input_Data(Clock, Reset, In_Data,
Out_Total);
input      Clock, Reset;
input  [7:0] In_Data;
output [7:0] Out_Total;
reg  [7:0] Out_Total;
reg  [1:0] state;// 因為採用二進制碼編碼方式，3 個狀態需 2 個正反器
                  (也就是 reg [1:0])
// 如果改用 One-Hot 碼編碼方式 (001, 010, 100)，則需 3 個正反器 (reg [2:0])
```

```
// 如果選用 (3, 2)-碼編碼方式 (011,101,110) ,
// 同樣需要 3 個正反器 (reg [2:0])
parameter s0 = 2'b00, s1 = 2'b01, s2 = 2'b10;

always @(posedge Clock)
begin
    if (Reset) // 電路重置
    begin
        state = s0; // 初始狀態為 s0
    end
    else
    begin
        case (state)
            s0: // 目前狀態為 s0
            begin
                Out_Total = In_Data;
                state = s1; // 下個狀態為 s1
            end
            s1: // 目前狀態為 s0
            begin
                Out_Total = Out_Total + In_Data;
                state = s2; // 下個狀態為 s2
            end
            default: // 其他狀態
            begin
                Out_Total = Out_Total + In_Data;
                state = s0; // 下個狀態為 s0
            end
        endcase
    end
end

endmodule
```



▲ Sum3Data.V 合成後，電路的模擬結果

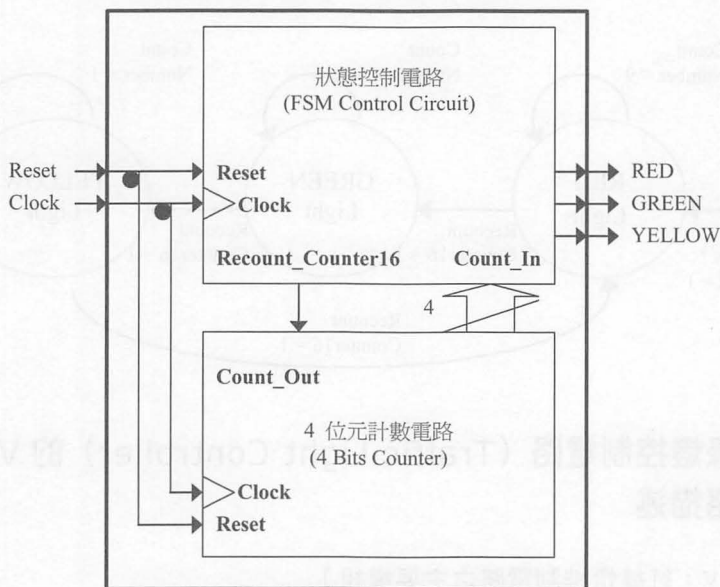


10.4 有限狀態機器的設計實例

使用 Verilog 硬體描述語言來實作數位系統的例子實在是非常多，在這一章節中就以紅綠燈控制電路這個例子來加以說明並作討論。

(一) 紅綠燈控制電路 (Traffic Light Controller) 的邏輯符號

整個「紅綠燈控制電路 (Traffic Light Controller) , Traffic.V」是由：「狀態控制電路 , Tra_Cntl.V」以及「4 位元計數電路 , Cnt16.V」所形成的。



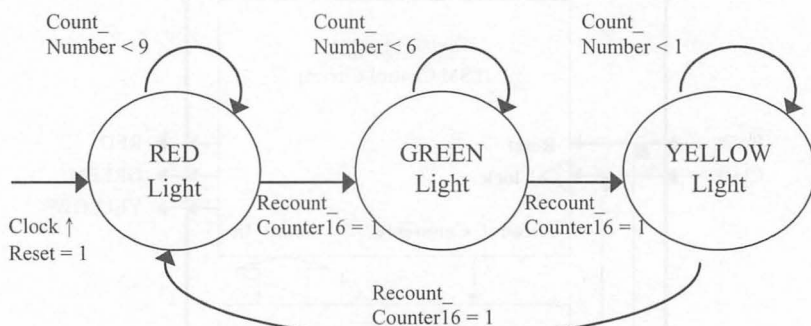
▲ 紅綠燈控制電路 (Traffic Light Controller) 的邏輯符號

當時脈 (Clock) 訊號來臨且有外部的 Reset 訊號時，「狀態控制電路」會進入初始狀態 (RED_Light, 紅燈亮)，且透過 Recount_Counter16 設為 1 的這個動作告知「4 位元計數電路」在下一個時脈 (Clock) 訊號來臨時將重新計數。

之後的每一個時脈 (Clock) 訊號時，「4 位元計數電路」內的計數值就 +1 而且把最新的計數值傳送給「狀態控制電路」；「狀態控制電路」會根據本身目前狀態以及由「4 位元計數電路」傳送過來的計數值去決定是否作狀態的改變。

例如：目前狀態為 RED_Light (紅燈)，而且已經過了 RED_Light_Time 個時間單位，則下個狀態將是 GREEN_Light (綠燈)，且透過設定 Counter_16 為 1 的方式告知在下一個時脈 (Clock) 訊號來臨時「4 位元計數電路」將重新計數。

(二) 狀態圖：紅綠燈控制電路 (Traffic Light Controller)



(三) 紅綠燈控制電路 (Traffic Light Controller) 的 Verilog 電路描述

【Traffic.V：紅綠燈控制電路之主要模組】

```

// Traffic.V：紅綠燈控制電路之主要模組

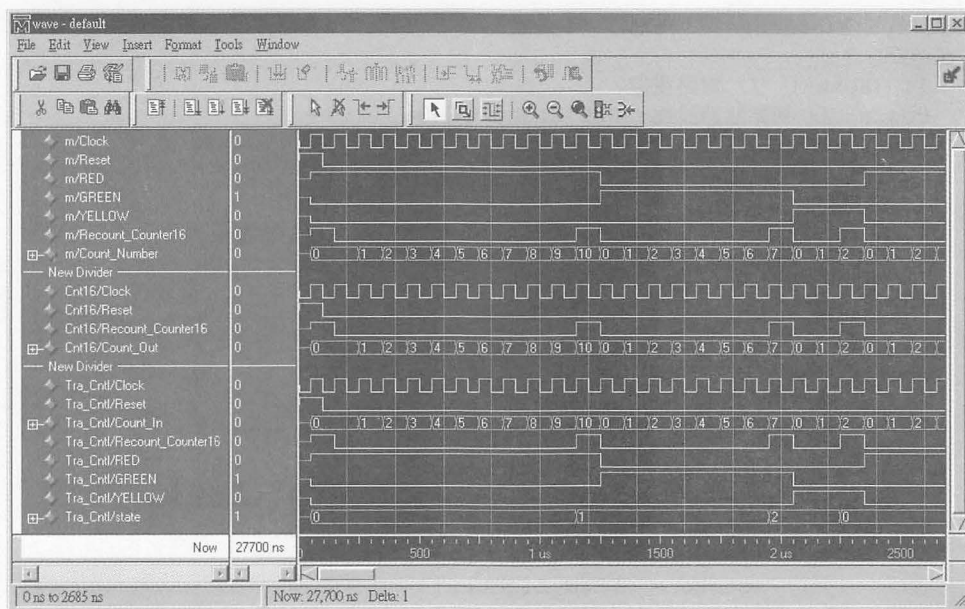
`include "Cnt16.V" // 引用 Cnt16.V, 4 位元計數器
`include "Tra_Cnt1.V" // 引用 Tra_Cnt1.V, 紅綠燈控制器的狀態控制電路

module Traffic (Clock, Reset, RED, GREEN, YELLOW);
input      Clock, Reset;
output     RED, GREEN, YELLOW;
  
```

```
// 下列的這些變數名稱 是用於內部的接線
wire      RED, GREEN, YELLOW;
wire      Recount_Counter16;
wire [3:0] Count_Number;

    Traffic_Control_FSM (Clock, Reset, Count_Number,
Recount_Counter16,
                        RED, GREEN, YELLOW);
    Counter_16 (Clock, Reset, Recount_Counter16, Count_Number);

endmodule
```



▲ Traffic.V 合成後，電路的模擬結果

【Tra_Cntl.V：紅綠燈控制器的狀態控制電路 (Traffic Light Controller，FSM)】

```
// Tra_Cntl.V：紅綠燈控制器的狀態控制電路 (Traffic Light Controller, FSM)

module Traffic_Control_FSM (Clock, Reset, Count_In,
                            Recount_Counter16,
                            RED, GREEN, YELLOW);
```

```
input      Clock, Reset;
input  [3:0] Count_In;
output     Recount_Counter16, RED, GREEN, YELLOW;
reg        Recount_Counter16, RED, GREEN, YELLOW;
reg  [1:0] state;

`define RED_Light_Time      9 // 紅燈亮的時間倍值
`define GREEN_Light_Time   6 // 紅燈亮的時間倍值
`define YELLOW_Light_Time  1 // 紅燈亮的時間倍值

parameter RED_Light      = 2'b00, GREEN_Light = 2'b01,
          YELLOW_Light = 2'b10;

always @(posedge Clock)
begin
    if (Reset) // 電路重置
    begin // 初始狀態設為: RED_Light (紅燈)
        state = RED_Light;
        RED    = 1'b1; // 紅燈亮
        GREEN  = 1'b0;
        YELLOW = 1'b0; // 重新計數 Counter_16 設為: 1 (重新計數)
        Recount_Counter16 = 1;
    end
    else
    begin
        case (state)
            RED_Light: // 目前狀態為: RED_Light (紅燈)
                begin // 紅燈: 維持 RED_Light_Time 個時間單位
                    if (Count_In == `RED_Light_Time)
                    begin // 重新計數 Counter_16 設為: 1 (重新計數)
                        Recount_Counter16 = 1;
                        // 下個狀態為: GREEN_Light (綠燈)
                        state = GREEN_Light;
                    end
                end
            else
                begin // 重新計數 Counter_16 設為: 0 (不重新計數)
                    Recount_Counter16 = 0;
                    RED    = 1'b1; // 紅燈亮
                    GREEN  = 1'b0;
                    YELLOW = 1'b0;
                end
            end
        end

        GREEN_Light: // 目前狀態為: GREEN_Light (綠燈)
            begin // 綠燈: 維持 GREEN_Light_Time 個時間單位
                if (Count_In == `GREEN_Light_Time)
```

```

begin // 重新計數 Counter_16 設為: 1 (重新計數)
    Recount_Counter16 = 1;
    // 下個狀態為: YELLOW (黃燈)
    state = YELLOW_Light;
end
else
begin // 重新計數 Counter_16 設為: 0 (不重新計數)
    Recount_Counter16 = 0;
    RED    = 1'b0;
    GREEN  = 1'b1; // 綠燈亮
    YELLOW = 1'b0;
end
end

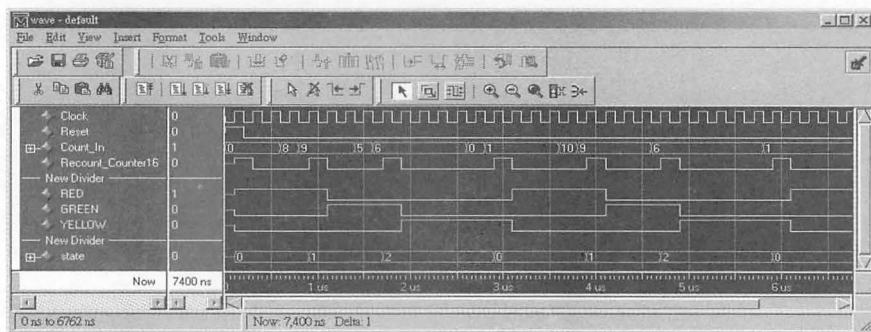
YELLOW_Light: // 目前狀態為: YELLOW_Light (黃燈)
begin // 黃燈: 維持 YELLOW_Light_Time 個時間單位
    if (Count_In == `YELLOW_Light_Time)
    begin // 重新計數 Counter_16 設為: 1 (重新計數)
        Recount_Counter16 = 1;
        // 下個狀態為: RED_Light (紅燈)
        state = RED_Light;
    end
    else
    begin // 重新計數 Counter_16 設為: 0 (不重新計數)
        Recount_Counter16 = 0;
        RED    = 1'b0;
        GREEN  = 1'b0;
        YELLOW = 1'b1; // 黃燈亮
    end
end

default: // 目前狀態為: 無效的的電路狀態 (X 或 = 2'b11)
begin
    state = 2'bXX; // 無效的的電路狀態 (2'bXX)
    // 重新計數 Counter_16 設為: 0 (不重新計數)
    Recount_Counter16 = 0;
    RED    = 1'bX;
    GREEN  = 1'bX;
    YELLOW = 1'bX;
end

endcase
end
end

endmodule

```



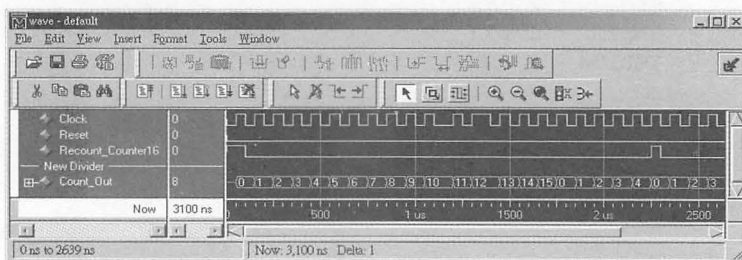
▲ Tra_Cntl.V 合成後，電路的模擬結果

【Cnt16.V: 4 位元計數器 (Traffic Light Controller, Counter)】

```
// Cnt16.V: 4 位元計數器 (Traffic Light Controller, Counter)
// 具有「重置」及「重新計數」訊號控制的
// 4 位元 (0~15) 正緣觸發計數器
```

```
module Counter_16 (Clock, Reset, Recount_Counter16, Count_Out);
input      Clock, Reset, Recount_Counter16;
output [3:0] Count_Out;
reg [3:0] Count_Out;

always @ (posedge Clock)
begin
    if (Reset) // 重置
        Count_Out = 0; // 清除為 0
    else
        if (Recount_Counter16) // 重新計數
            Count_Out = 0; // 清除為 0
        else
            Count_Out = Count_Out + 1; // 計數值 +1
end
endmodule
```



▲ Cnt16.V 合成後，電路的模擬結果

本章練習**Question 1 :**

請設計一個公車投幣機(台北聯營公車每段票：全票 15 元、學生票 12 元與優待票 8 元)的有限狀態機器電路。

Question 2 :

請設計一個捷運兌票機(高雄捷運票價：單程票 20~60 元、間隔 5 元)的有限狀態機器電路。

Question 3 :

請設計一台吃角子老虎(Slots)拉霸的有限狀態機器電路。

PS Google 可以幫您找到解答。

